Dialektly Testkonzept

Schweizerdeutsch-Vokabeltrainer



Projektteam	Team03 (Sven Scheuss, Raphael Stricker, Lena Wöhrle & Joris Neber)

1 Einleitung und Testziele

Dieses Testkonzept beschreibt die Teststrategie zur Sicherstellung der funktionalen Korrektheit der Dialektly-Applikation. Das Ziel ist die systematische Verifikation der geschaeftskritischen Logik. Die Benutzeroberflaeche (JavaFX/FXML) ist explizit aus dem Testscope ausgenommen. Der Fokus liegt auf Unit- und Integrationstests der Model-Schichten.

2 Teststrategie

2.1 Testumfang

Getestet werden die zentralen Domänenklassen:

- **LearningSession**
  - Session-Lebenszyklus
  - Queue-Management
  - Fortschrittslogik
- **Exercise (Interface / abstrakte Klasse)**
  - Antwortvalidierung (submitAnswer, isCorrect, isMastered)
- **UserProgress**
  - Fortschrittsberechnung über Topics hinweg (markLearned, getOverallPercentage, isCompleted)
- **TopicProgress**
  - Fortschritt pro Topic (incrementLearned, getPercentage, isCompleted)

Nicht getestete Klassen:

- UI-Klassen (JavaFX Controller, FXML)
  -> Nicht Teil des Testscopes, da Fokus auf Business-Logik
- Reine Datenklassen (z. B. VocabularyItem oder User, einfache DTOs)
  -> Enthalten keine Logik, daher kein Mehrwert durch Unit-Tests
- Navigator / Presenter
  -> JavaFX-UI-Schicht, explizit aus dem Testscope ausgenommen.
- Enums (AppLanguage, DialectRegion, TopicCategory, ExerciseType)
  -> Keine Logik, keine Tests nötig.

2.2 Testumsetzung

Unit-Tests:

- Fokus: isolierte Tests einzelner Klassen
- Mocking von Abhängigkeiten mit Mockito
- Ziel: deterministisches Verhalten prüfen

Integrationstests:

- Zusammenspiel mehrerer Komponenten
- z. B. LearningSession + Exercise

Testframework: JUnit 5 + Mockito. Äquivalenzklassenanalyse und Grenzwertanalyse werden für alle Servicemethoden angewendet.

Wer testet? Das Entwicklungsteam (Team 03 / NullPointerNinjas) ist für die Durchführung aller Tests verantwortlich. Unit-Tests werden von der jeweiligen Person geschrieben, die die entsprechende Komponente entwickelt hat. Integrationstests werden gemeinsam im Team reviewed und abgenommen.

Wann wird getestet? Tests werden parallel zur Entwicklung erstellt und ausgeführt. Unit-Tests begleiten jede Implementierung kontinuierlich; Integrationstests folgen nach Fertigstellung jeder Komponente.

2.3 Testumgebung und Werkzeuge

- IDE: IntelliJ
- Sprache & Plattform: Java 21, Gradle Build
- Test-Framework: JUnit 5
- Mocking: Mockito
- Integrationstests: Temporäre Verzeichnisse via @TempDir (Junit 5)

3 Äquivalenzklassenanalyse

+ submitAnswer(answer: String): Boolean – Klasse: LearningSession						
AeK-ID	**Name**	**answer : String**	**LearningSession.queue**	**LearningSession.masteredItems**	**Behavior (Rückgabe)**	**Nachzustand**
AeK-01	Korrekte Antwort – erstmals richtig	Korrekte Antwort (exakt)	Exercise in queue, correctCount = 0	masteredItems enthält Item nicht	Gibt true zurück; correctCount wird erhöht (→1)	correctCount = 1; Exercise bleibt in queue
AeK-02	Korrekte Antwort – Exercise gemeistert	Korrekte Antwort (exakt)	Exercise in queue, correctCount = 1	masteredItems enthält Item nicht	Gibt true zurück; Exercise in masteredItems	masteredItems +1; queue −1; isMastered() = true
AeK-03	Korrekte Antwort nach Normalisierung	Korrekte Antwort mit abweichender Gross-/Kleinschreibung (z.B. 'Öpfel')	Exercise in queue, correctCount = 0	masteredItems enthält Item nicht	Normalisierung erkannt; gibt true zurück; correctCount → 1	correctCount = 1; queue unverändert
AeK-04	Falsche Antwort	Inhaltlich falsche Antwort	Exercise in queue, beliebiger correctCount	Beliebig	Gibt false zurück; Exercise verbleibt in queue	queue unverändert; correctCount unverändert
AeK-05	Leere Antwort (Leerstring)	""	Exercise in queue, beliebiger correctCount	Beliebig	Gibt false zurück (keine Exception)	queue unverändert
AeK-06	Null-Antwort	null	Beliebig	Beliebig	IllegalArgumentException oder NullPointerException; kein Absturz ohne Exception-Handling	queue und masteredItems unverändert

+ isCompleted(): boolean – Klasse: LearningSession					
AeK-ID	**Name**	**queue : Deque<Exercise>**	**masteredItems : Set<VocabularyItem>**	**Behavior (Rückgabe)**	**Nachzustand**
AeK-07	Session abgeschlossen – alle Vokabeln gemeistert	queue ist leer	enthält alle Items	Gibt true zurück	Kein State verändert
AeK-08	Session läuft – noch Exercises ausstehend	queue enthält mind. 1 Exercise	unvollständig	Gibt false zurück	Kein State verändert
AeK-09	Session läuft – queue leer aber masteredItems unvollständig	queue ist leer	enthält nicht alle Items	Gibt false zurück (Inkonsistenz-Schutz)	Kein State verändert

+ LearningSession(topic: TopicCategory, items: List<VocabularyItem>, type: ExerciseType) — Konstruktor						
AeK-ID	**Name**	**topic : TopicCategory**	**items : List<VocabularyItem>**	**type: ExerciseType**	**Behavior**	**Nachzustand**
AeK-10	Gültige Session erstellt	Gültige TopicCategory (z.B. ALLTAG)	Nicht-leere Liste mit VocabularyItems	TEXT_INPUT	LearningSession-Objekt erstellt; queue befüllt; sessionId generiert	queue.size() = items.size(); getMasteredItems leer; isCompleted() = false
AeK-11	Leere Vokabelliste	Gültige TopicCategory	Leere Liste (size = 0)	TEXT_INPUT	LearningSession erstellt; queue leer; isCompleted() = true (da keine Items)	queue.size() = 0; isCompleted() = true
AeK-12	Fehler – topic null	null	Beliebige Liste	Beliebig	NullPointerException oder IllegalArgumentException	Kein Objekt erstellt
AeK-13	Fehler – items null	Gültige TopicCategory	null	Beliebig	NullPointerException oder IllegalArgumentException	Kein Objekt erstellt
AeK-14	type TEXT_TO_VOICE	Gueltige TopicCategory	Nicht-leere Liste	TEXT_TO_VOICE	UnsupportedOperationException	Kein Objekt erstellt

+ submitAnswer(answer: String): boolean – Interface: Exercise						
AeK-ID	**Name**	**Typ**	**answer**	**correctCount vorher**	**Behavior**	**Nachzustand**
AeK-15	Richtige Antwort – TEXT_INPUT	TextInputExercise	standardWord (exakt)	0	Gibt true; correctCount -> 1	correctCount = 1; isMastered() = false
AeK-16	Richtige Antwort – gemeistert	TextInputExercise	standardWord (exakt)	1	Gibt true; correctCount -> 2	isMastered() = true
AeK-17	Richtige Antwort – AUDIO_TO_TEXT	AudioToTextExercise	dialectWord (exakt)	0	Gibt true; correctCount -> 1	correctCount = 1
AeK-18	Falsche Antwort	TextInputExercise	Falsche Antwort	1	Gibt false; correctCount -> 0 (reset)	correctCount = 0; isMastered() = false
AeK-19	Case-insensitive korrekt	TextInputExercise	standardWord (andere Gross-/Kleinschreibung)	0	Gibt true	correctCount = 1
AeK-20	Leere Antwort	TextInputExercise	""	0	Gibt false	correctCount = 0
AeK-21	Null-Antwort	TextInputExercise	null	0	Gibt false (kein Crash)	correctCount = 0

UserProgress – Fortschrittsberechnung				
AeK-ID	**Name**	**Ausgangszustand**	**Aktion**	**Erwartetes Ergebnis**
AeK-22	Kein Fortschritt (0%)	Keine Topics initialisiert	getOverallPercentage()	0.0
AeK-23	Teilfortschritt ein Topic	ALLTAG: 5/10 gelernt	getOverallPercentage()	50.0
AeK-24	Mehrere Topics unabhaengig	ALLTAG: 10/10, ARBEIT: 0/10	getOverallPercentage()	50.0 (Durchschnitt)
AeK-25	Alles abgeschlossen	Alle Topics 100%	isCompleted()	true
AeK-26	Teilweise abgeschlossen	Mind. 1 Topic < 100%	isCompleted()	false
AeK-27	markLearned doppelt	Item bereits in learnedItems	markLearned(selbes Item)	learnedCount unveraendert (kein Duplikat)
AeK-28	markLearned – kein TopicProgress	Topic nicht initialisiert	markLearned(item)	TopicProgress automatisch erstellt; learnedCount = 1

TopicProgress – incrementLearned / getPercentage / isCompleted					
AeK-ID	**Name**	**totalWords**	**learnedCount**	**Aktion**	**Erwartetes Ergebnis**
AeK-29	0% Fortschritt	10	0	getPercentage()	0.0
AeK-30	Teilfortschritt	10	5	getPercentage()	50.0
AeK-31	100% abgeschlossen	10	10	isCompleted()	true
AeK-32	Noch nicht fertig	10	9	isCompleted()	false
AeK-33	Ueberlauf verhindern	10	10	incrementLearned()	learnedCount bleibt 10 (kein Overflow)
AeK-34	totalWords = 0	0	0	getPercentage()	0.0 (kein Division-by-zero)
AeK-35	totalWords = 0, isCompleted	0	0	isCompleted()	false (leeres Topic gilt nicht als done)